Chapter 11 — TypeScript with React

TypeScript is JavaScript plus a type system (a set of labels saying what kind of value each variable holds — a string, a number, an object of a certain shape — so mismatches can be caught) that is checked **before** your code runs. In a React project it turns a whole class of bugs — a missing prop, a misspelled variant, reading `user.name` while `user` can still be `null`, forgetting to handle one action in a reducer (a function that takes the current state plus an "action" describing what happened and returns the next state) — into red squiggles in your editor instead of broken screens in production (the live app that real users run). It also makes components self-documenting: the props type *is* the component's API (its public interface: what callers may pass in and get back).

This chapter assumes the TypeScript basics from the JavaScript Handbook (Chapter 16 — types, narrowing, generics) and shows how to apply them to React: props (the inputs a parent passes to a component), children, events, state, reducers, refs (boxes that hold a DOM element or a value across renders), Hooks (the `use...` functions such as `useState` that give components React features), context (a way to hand a value to deeply nested components without passing props through every level), generic components and discriminated-union props. For the "wrong" examples you will see the **real compiler errors** (messages from the TypeScript compiler, the program that reads and checks your code).

> **Key term — narrowing and generics:** *Narrowing* means TypeScript shrinks a broad type to a more specific one after a check — after `if (user !== null)`, `user` is known to be a `User`. *Generics* are types with a placeholder, written `<T>`, that gets filled in where they are used — `Array<string>` is an array whose items are strings — so one piece of code can work for many types and still be checked.

What you'll learn

#	Topic	You will be able to…
11.1	Setup and how type-checking fits in	create a React + TS project and run `tsc --noEmit`
11.2	Typing props	write props types with required, optional, union and callback props
11.3	Typing children and extending native elements	choose between `ReactNode`, `ReactElement`, `PropsWithChildren`, `ComponentProps<'button'>`
11.4	Typing events	type handlers with `ChangeEvent`, `SubmitEvent`, `MouseEvent`, `KeyboardEvent`
11.5	Typing state and reducers	type `useState` (including `null` and `[]`) and `useReducer` with action unions
11.6	Typing refs	type DOM refs, value refs, `ref` as a prop (React 19) and `useImperativeHandle`
11.7	Typing custom Hooks and Context	return tuples `as const`, write generic Hooks, make context non-nullable
11.8	Generic components	build `List<T>`, `Select<V>` and `Table<Row>` that infer their types
11.9	Discriminated unions for props	make impossible prop combinations impossible to write
11.10	Reading type errors, and common mistakes	decode the 12 errors beginners hit most

Code: `code/`. Each `.tsx` file **runs** (bundled with esbuild — a very fast bundler; React 19.3 running in jsdom, a JavaScript imitation of a browser's DOM, the Document Object Model tree of page elements, that runs in Node.js, the program that runs JavaScript outside a browser) **and** is type-checked with the TypeScript compiler (`tsc` 7.0.2, `@types/react` 19.3, `strict` mode — the setting that turns on all the stricter checks). The deliberately broken examples live in `code/bad_types.tsx`, which is only type-checked.

11.1 Setup and how type-checking fits in

What it is

A React + TypeScript project has three pieces:

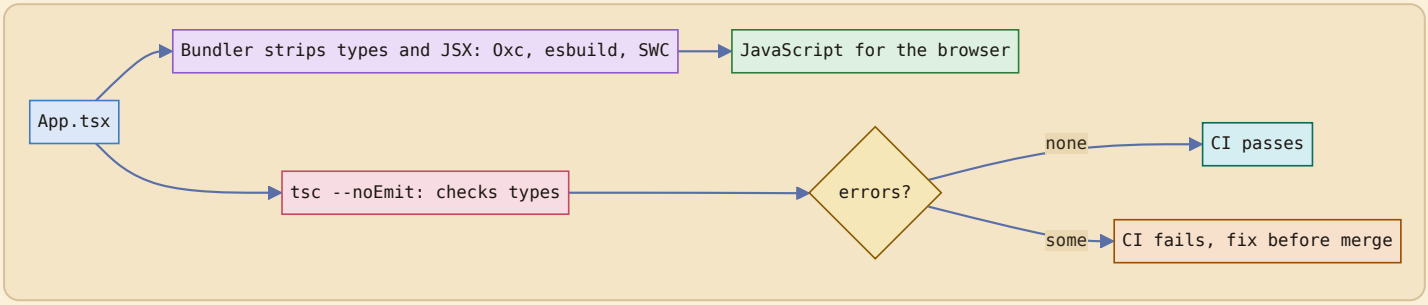
Piece	Role
`.tsx` files	TypeScript files that may contain JSX (HTML-like tags in code; `.ts` for files without JSX)
`@types/react`, `@types/react-dom`	the type definitions for React (React itself is written in JavaScript; these packages describe its API)
`tsconfig.json`	compiler options: strictness, JSX mode, which files to check

To start a new project: `npm create vite@latest my-app -- --template react-ts` (npm is Node's package manager, the tool that downloads libraries; Vite is a fast development server and build tool), or a framework (a library that also decides your project's structure) such as Next.js, a React framework with routing and server rendering built in, which asks whether you want TypeScript and configures it (Chapter 1).

How it works — two separate jobs

1. **Removing types.** Browsers can't run TypeScript, so the bundler (the tool that combines your source files and libraries into files a browser can load; Vite 8 uses Oxc, earlier Vite versions esbuild; Next.js uses SWC — Oxc and SWC are fast compilers written in Rust) strips the types and turns JSX into function calls. This is fast because it **does not check** anything.
2. **Checking types.** The TypeScript compiler, `tsc`, reads all files and reports errors. With `--noEmit` it only checks and writes no files. Your editor runs the same checker continuously.

Because the bundler does not check types, **a project with type errors can still build and run**. Add `tsc --noEmit` to your CI (continuous integration: a server that automatically runs checks and tests on every push or pull request; and often to `npm run build`, e.g. `"build": "tsc --noEmit && vite build"`) so errors block merges (accepting a change into the main branch of the code).



The tsconfig options that matter for React

`code/tsconfig.check.json` (this book keeps npm packages in a shared folder, hence the `paths` line; a normal project doesn't need it and names the file `tsconfig.json`):

```jsonc
{
  "compilerOptions": {
    "target": "ES2022",
    "module": "ESNext",
    "moduleResolution": "bundler",     // resolve imports the way Vite/esbuild do
    "jsx": "react-jsx",                // the modern JSX transform: no `import React` needed
    "strict": true,                    // turns on strictNullChecks, noImplicitAny, ... - always use it
    "noEmit": true,                    // the bundler produces JS; tsc only checks
    "skipLibCheck": true,              // don't re-check .d.ts files in node_modules (faster)
    "allowImportingTsExtensions": true,
    "lib": ["ES2023", "DOM", "DOM.Iterable"],   // DOM types such as HTMLInputElement
    "types": [],
    "paths": { "*": ["../../../../.web-books/node_modules/@types/*", "../../../../.web-books/node_modules/*"] }
  },
  "include": ["*.tsx", "*.ts"],
  "exclude": ["bad_*.tsx"]
}
```

Checking every example in this chapter:

```
$ tsc --noEmit -p tsconfig.check.json && echo "0 type errors"
0 type errors
```

Common mistakes

- Turning `strict` off to "get it compiling" — you lose most of the value, especially null checks.
- Using `any` (the type that means "anything, don't check") to silence an error. `any` switches checking off for that value *and everything it flows into*. Prefer `unknown` ("some value, type not known yet" — you must check it before using it) and narrow it, or fix the type.
- Forgetting `@types/react` / `@types/react-dom` (or having versions that don't match your React major version — the first number in `19.3.0`, which changes when there are breaking changes).

TL;DR

Bundlers strip types; `tsc --noEmit` checks them. Use `strict: true`, `jsx: "react-jsx"`, and run the checker in CI.

11.2 Typing props

What it is

A component's props are one object, so you describe that object with a `type` (or `interface`) and annotate the destructured parameter (write `: Props` after the `{ label, count }` pattern that unpacks the props object by name).

`code/01_props_and_children.tsx` (part 1):

```tsx
type BadgeProps = {
  label: string;                        // required
  count?: number;                       // optional -> number | undefined inside the component
  tone?: 'info' | 'warning' | 'error';  // only these three strings are allowed
};
function Badge({ label, count = 0, tone = 'info' }: BadgeProps) {
  return <span className={`badge ${tone}`}>{label}: {count}</span>;
}

type AvatarProps = {
  user: { name: string; imageUrl: string };  // an object shape
  size?: number;
  style?: CSSProperties;                     // a style object, checked key by key
  onSelect?: (name: string) => void;         // a callback prop
};
function Avatar({ user, size = 32, style, onSelect }: AvatarProps) {
  return <img src={user.imageUrl} alt={user.name} width={size} style={style} onClick={() => onSelect?.(user.name)}
/>;
}
```

How it works

1. At each `<Badge ...>` usage, TypeScript checks the JSX attributes against `BadgeProps`, exactly like checking an object literal.
2. A `?` makes the prop optional; inside the component its type includes `undefined` until you give a **default value** in the destructuring (`count = 0` makes `count` a plain `number`).
3. A **union of string literals** (`'info' | 'warning' | 'error'`) is the TypeScript way to say "one of these values" — editors autocomplete them, and typos are errors.
4. A callback prop (a function the parent passes in for the child to call later) is a **function type**: `(name: string) => void` says "called with a string, return value ignored". Call optional callbacks with `onSelect?.(...)` (optional chaining: call it only if it is not `undefined`).

`type` or `interface`?

Both work for props. `interface` can be extended with `extends` and merged by re-declaring; `type` can express unions (`A | B`), which props often need (§11.9). Many teams simply use `type` everywhere for consistency; pick one convention per codebase.

What about `React.FC`?

`const Badge: React.FC<BadgeProps> = (props) => ...` also works. Since React 18's types, `FC` no longer adds an implicit `children` prop, so the old main objection is gone; still, annotating the props parameter directly (as above) is simpler, works with generics (§11.8) and is what most modern code does.

Real-world examples

Design-system components (buttons, badges, alerts) use literal unions for `variant`/`size`/`tone` props, so the design tokens (the named, agreed values for colours, spacing and sizes in a design system — a company's shared set of UI components and rules) are enforced by the compiler.

TL;DR

`function C({ a, b = 1 }: Props)`. `?` = optional, defaults remove `undefined`, literal unions restrict values, function types describe callbacks.

11.3 Typing children and extending native elements

Which type for `children`?

Type	Accepts	Use for
`ReactNode`	anything React can render: JSX, strings, numbers, `null`, `undefined`, booleans, arrays of these	**`children` and "slot" props** — the default choice
`ReactElement`	only a JSX element (`<Foo />`), not strings or `null`	a prop that must be one element (rare)
`JSX.Element`	the type of a JSX expression (basically `ReactElement<any, any>`)	the return type of components, if you annotate it
`PropsWithChildren<P>`	adds `children?: ReactNode` to `P`	a short way to accept optional children

`code/01_props_and_children.tsx` (part 2):

```tsx
type CardProps = { title: string; children: ReactNode; footer?: ReactNode };
function Card({ title, children, footer }: CardProps) {
  return <section><h2>{title}</h2>{children}{footer && <footer>{footer}</footer>}</section>;
}
function Note({ children }: PropsWithChildren) { return <aside>{children}</aside>; }
```

Extending a native element's props

A reusable `<Button>` should accept **everything** a `<button>` accepts — `type`, `disabled`, `aria-*` (accessibility attributes that screen readers announce), `onClick` — without you listing them. `ComponentProps<'button'>` is the full props type of the built-in element:

```tsx
type ButtonProps = ComponentProps<'button'> & { variant?: 'primary' | 'ghost' };
function Button({ variant = 'primary', className, ...rest }: ButtonProps) {
  return <button className={[variant, className].filter(Boolean).join(' ')} {...rest} />;
}
```

Usage and result:

```tsx
<Card title="Inbox" footer={<small>updated now</small>}>
  <Badge label="unread" count={3} tone="warning" />
  <Note>plain text child</Note>
  <Button type="submit" disabled aria-label="send">Send</Button>
  <Avatar user={{ name: 'Ada', imageUrl: '/ada.png' }} style={{ borderRadius: '50%' }} />
</Card>
```

Output:

```
<section>
<h2>Inbox</h2>
<span class="badge warning">unread: 3</span>
<aside>plain text child</aside>
<button class="primary" type="submit" disabled="" aria-label="send">Send</button>
<img alt="Ada" width="32" style="border-radius: 50%;" src="/ada.png">
<footer>
<small>updated now</small>
</footer>
</section>
```

Related helpers: `ComponentProps<typeof MyComponent>` extracts the props type of **your** component (handy when wrapping a library component whose props type isn't exported); `ComponentPropsWithoutRef<'input'>` omits `ref`; `Omit<ComponentProps<'a'>, 'href'>` removes one prop so you can redefine it.

Common mistakes

- Typing `children` as `JSX.Element` — then `<Card>hello</Card>` (a string) or two children fail. Use `ReactNode`.
- Spreading `{...rest}` (the spread syntax copies every remaining property onto the element) onto the element *before* your own props, so the caller's `className` overwrites yours. Merge deliberately, as above.

TL;DR

`children: ReactNode`; extend native props with `ComponentProps<'button'> & {...}` and spread the rest.

11.4 Typing events

What it is

React passes your handlers its own event objects (synthetic events, Chapter 3). Their types are generic over the **element** the handler is attached to, which is what makes `e.target.value` or `e.currentTarget.checked` type-safe (misuse is a compile error instead of a runtime crash).

Event	Type	Typical element	Common props
`onChange` on input / select / textarea	`ChangeEvent<HTMLInputElement>` (etc.)	`HTMLInputElement`, `HTMLSelectElement`, `HTMLTextAreaElement`	`e.target.value`, `e.target.checked`
`onSubmit`	`SubmitEvent<HTMLFormElement>` (older code: `FormEvent`)	`HTMLFormElement`	`e.preventDefault()`, `e.submitter`
`onClick`	`MouseEvent<HTMLButtonElement>`	any element	`e.currentTarget`, `e.shiftKey`, `e.clientX`
`onKeyDown`	`KeyboardEvent<HTMLInputElement>`	any focusable element	`e.key`, `e.ctrlKey`
`onFocus` / `onBlur`	`FocusEvent<HTMLInputElement>`		`e.relatedTarget`

Import them from `react`, **not** from the DOM (`MouseEvent` or `SubmitEvent` without an import refers to the browser's native event type, which is different).

Version note: recent `@types/react` 19 releases (used here: 19.3) type `onSubmit` with `SubmitEvent` and mark `FormEvent` as **deprecated** (still works, but flagged as outdated and likely to be removed; React never had a real "form event"). `FormEvent<HTMLFormElement>` still compiles, so you'll see it in most existing code and tutorials; prefer `SubmitEvent` in new code if your `@types/react` has it.

How it works

1. **Inline handlers need no annotation**: in `onChange={e => setEmail(e.target.value)}`, TypeScript already knows the `onChange` prop's type, so `e` is inferred (its type is worked out automatically) (this is called *contextual typing*).
2. **Separately declared handlers need a type**, because TypeScript can't see where they'll be used. Either annotate the parameter (`e: ChangeEvent<HTMLInputElement>`) or the whole function (`const h: ChangeEventHandler<HTMLSelectElement> = e => ...`).
3. Tip: not sure of the type? Write the handler inline first and hover `e` in your editor; it shows the exact type to copy.

`code/02_events.tsx`:

```tsx
function SignupForm({ onSubmit }: { onSubmit: (data: { email: string; plan: string }) => void }) {
  const [email, setEmail] = useState('');
  const [plan, setPlan] = useState('free');

  function handleEmail(e: ChangeEvent<HTMLInputElement>) {
    setEmail(e.target.value);                       // e.target is an HTMLInputElement -> .value is a string
  }
  const handlePlan: ChangeEventHandler<HTMLSelectElement> = e => setPlan(e.target.value); // type the whole function
  function handleSubmit(e: SubmitEvent<HTMLFormElement>) {   // older code: FormEvent (now deprecated)
    e.preventDefault();
    console.log('submitted by the button:', e.submitter?.textContent);
    onSubmit({ email, plan });
  }
  function handleKey(e: KeyboardEvent<HTMLInputElement>) {
    if (e.key === 'Escape') setEmail('');
  }
  function handleHelp(e: MouseEvent<HTMLButtonElement>) {
    console.log(`help clicked at button "${e.currentTarget.textContent}" (shift: ${e.shiftKey})`);
  }

  return (
    <form onSubmit={handleSubmit}>
      <input aria-label="email" value={email} onChange={handleEmail} onKeyDown={handleKey} />
      <select aria-label="plan" value={plan} onChange={handlePlan}>
        <option value="free">Free</option><option value="pro">Pro</option>
      </select>
      <button type="button" onClick={handleHelp}>Help</button>
      <button type="submit" onClick={e => console.log('inline handler, e is', e.type)}>Sign up</button>
    </form>
  );
}
```

Output (after typing an email, choosing Pro, shift-clicking Help, submitting, then pressing Escape):

```
help clicked at button "Help" (shift: true)
inline handler, e is click
submitted by the button: Sign up
submitted {"email":"ada@example.com","plan":"pro"}
after Escape, email = ""
```

`target` vs `currentTarget`

`e.currentTarget` is the element the handler is attached to, and its type is the element type you gave (`HTMLButtonElement`). `e.target` is whatever element was actually clicked (maybe an `<svg>` inside the button), so for most events it is typed only as `EventTarget`. For **`change`** events React types `e.target` as the input itself, which is why `e.target.value` works there.

Form actions (React 19)

With form actions (Chapter 4) you don't handle an event at all: `<form action={fn}>` calls `fn(formData: FormData)` (`FormData` is the browser's object holding a form's field names and values). `formData.get('email')` returns `FormDataEntryValue | null` (a `string` or a `File`), so narrow it: `const email = String(formData.get('email') ?? '')`, or validate the whole object with a schema library such as Zod (a schema is a description of the expected data shape; Zod checks real data against it).

> **TL;DR**
>
> Inline handlers are inferred; standalone ones use `XxxEvent<HTMLElementType>` or `XxxEventHandler<...>` from `react`. `currentTarget` has your element's type.

11.5 Typing state and reducers

`useState`

TypeScript **infers** the state type from the initial value, so most calls need nothing extra. Add a type argument when the initial value doesn't show the full picture:

Code	Inferred / declared type	Why
`useState(0)`	`number`	inferred
`useState<User | null>(null)`	`User | null`	`null` alone would infer type `null` forever
`useState<string[]>([])`	`string[]`	`[]` alone infers `never[]` — you could never add anything
`useState<'idle' | 'saving' | 'saved'>('idle')`	the union	`'idle'` alone infers `string`
`useState(() => loadDraft())`	return type of `loadDraft`	lazy initializer (a function run only on the first render) is inferred too

`code/03_state_and_reducer.tsx` (part 1):

```
type User = { id: number; name: string };

function Profile() {
  const [count, setCount] = useState(0);                    // inferred: number
  const [user, setUser] = useState<User | null>(null);      // explicit: starts null, later a User
  const [tags, setTags] = useState<string[]>([]);           // [] alone would be never[]
  const [status, setStatus] = useState<'idle' | 'saving' | 'saved'>('idle');  // a union of literals

  return (
    <div>
      <p>{user ? `${user.name} (#${user.id})` : 'signed out'} | {count} | {tags.join(',')} | {status}</p>
      {/* user.name without the check would not compile: 'user' is possibly 'null' */}
      <button onClick={() => { setUser({ id: 7, name: 'Ada' }); setCount(c => c + 1); setTags(t => [...t,
'admin']); setStatus('saved'); }}>
        sign in
      </button>
    </div>
  );
}
```

The setter's type is `Dispatch<SetStateAction<T>>`: it accepts either a `T` or an updater `(prev: T) => T` (a function that computes the new state from the previous one). You need that name when passing a setter as a prop: `setUser: Dispatch<SetStateAction<User | null>>` (or simply `(u: User | null) => void` if the child never uses the updater form).

`useReducer` with a discriminated union of actions

A **discriminated union** is a union of object types that share one literal-typed field (the *discriminant*, here `type`). Checking that field **narrows** the object to one member, so TypeScript knows which other fields exist.

`code/03_state_and_reducer.tsx` (part 2):

```ts
type CartItem = { sku: string; qty: number };
type CartState = { items: CartItem[]; coupon: string | null };
type CartAction =
  | { type: 'add'; sku: string }
  | { type: 'remove'; sku: string }
  | { type: 'applyCoupon'; code: string }
  | { type: 'clear' };

function cartReducer(state: CartState, action: CartAction): CartState {
  switch (action.type) {
    case 'add': {
      const found = state.items.find(i => i.sku === action.sku);   // action.sku is known to exist here
      return {
        ...state,
        items: found
          ? state.items.map(i => (i.sku === action.sku ? { ...i, qty: i.qty + 1 } : i))
          : [...state.items, { sku: action.sku, qty: 1 }],
      };
    }
    case 'remove':      return { ...state, items: state.items.filter(i => i.sku !== action.sku) };
    case 'applyCoupon': return { ...state, coupon: action.code };  // action.code exists only here
    case 'clear':       return { items: [], coupon: null };
    default: {
      const unreachable: never = action;                          // compile error if a case is missing
      return unreachable;
    }
  }
}

function Cart() {
  const [cart, dispatch] = useReducer(cartReducer, { items: [], coupon: null });
  // dispatch accepts only CartAction objects: dispatch({ type: 'add', sku: 'MUG' })
  // ...
}
```

Output (sign in, add mug twice, apply coupon, then clear):

```
initial        signed out | 0 |  | idle  ||  empty coupon=none
after clicks   Ada (#7) | 1 | admin | saved  ||  MUGx2 coupon=SAVE10
after clear    Ada (#7) | 1 | admin | saved  ||  empty coupon=none
```

How the `never` check works

In the `default` branch, TypeScript has removed every handled member from `action`'s type. If all cases are handled, nothing is left — the type is `never`, and assigning it to `never` is fine. Add a new action type to `CartAction` and forget its `case`, and `action` in `default` still has that type, so the assignment **fails to compile** (you'll see this error in §11.10). This is called an **exhaustiveness check**.

`useReducer` infers the state type from the reducer, and `dispatch` accepts only valid actions. (React 19's types also let you write `useReducer<CartState, [CartAction]>(...)`, but annotating the reducer function is simpler and usually enough.)

> **TL;DR**
>
> Let `useState` infer; add `<T>` for `null`, `[]` and literal unions. Type reducer actions as a discriminated union and add a `never` check so missing cases are compile errors.

11.6 Typing refs

DOM refs and value refs

Purpose	Declaration	`.current` type
Point at a DOM element	`useRef<HTMLInputElement>(null)`	`HTMLInputElement | null`
Hold a mutable (changeable) value (timer ID, previous value)	`useRef<number | null>(null)`, `useRef(0)`	what you declared

In React 19's types, `useRef` **always requires an argument** and always returns a `RefObject<T>` whose `current` you can assign (older versions distinguished a read-only `RefObject` from a `MutableRefObject`; `MutableRefObject` is now deprecated). For DOM refs pass `null`, because the element doesn't exist until React attaches it after rendering — which is why you use `ref.current?.focus()`.

`ref` as a prop (React 19)

Since React 19, function components receive `ref` as an **ordinary prop**; `forwardRef` is no longer needed for new code (Chapter 5). `ComponentProps<'input'>` already includes `ref?: Ref<HTMLInputElement>`, so a wrapper that spreads its props forwards the ref automatically. For a custom handle (a small object of methods you choose to expose to the parent instead of the raw DOM node), type the prop as `Ref<YourHandle>` and fill it with `useImperativeHandle`.

`code/04_refs.tsx`:

```tsx
// 1. A DOM ref
function SearchBox() {
  const inputRef = useRef<HTMLInputElement>(null);
  useEffect(() => {
    inputRef.current?.focus();                  // ?. because current is null until React attaches it
  }, []);
  return <input ref={inputRef} aria-label="search" />;
}

// 2. A value ref (not attached to JSX)
function Stopwatch() {
  const intervalId = useRef<ReturnType<typeof setInterval> | null>(null);  // read/written only in handlers
  const [running, setRunning] = useState(false);
  function toggle() {
    if (intervalId.current !== null) { clearInterval(intervalId.current); intervalId.current = null;
setRunning(false); }
    else { intervalId.current = setInterval(() => {}, 1000); setRunning(true); }
  }
  return <button onClick={toggle}>{running ? 'stop' : 'start'} timer</button>;
}

// 3. ref as an ordinary prop - ComponentProps<'input'> includes `ref`
function TextField({ label, ...inputProps }: { label: string } & ComponentProps<'input'>) {
  return <label>{label} <input {...inputProps} /></label>;   // ref travels inside inputProps
}

// 4. Exposing a custom handle instead of the DOM node
export type VideoHandle = { play: () => void; pause: () => void };
function Video({ ref, src }: { ref?: Ref<VideoHandle>; src: string }) {
  const [state, setState] = useState('paused');
  useImperativeHandle(ref, () => ({ play: () => setState('playing'), pause: () => setState('paused') }), []);
  return <p>video {src}: {state}</p>;
}

function App() {
  const nameRef = useRef<HTMLInputElement>(null);
  const videoRef = useRef<VideoHandle>(null);
  // ... <TextField label="Name" ref={nameRef} placeholder="Ada" />
  //     <Video ref={videoRef} src="intro.mp4" />
  //     <button onClick={() => { ...; videoRef.current?.play(); }}>go</button>
}
```

Output:

```
focused after mount: search
stopwatch: stop timer
nameRef.current is INPUT with placeholder Ada
video intro.mp4: playing
```

`ReturnType<typeof setInterval>` is a trick worth knowing: the timer ID is a `number` in browsers but an object in Node, and this expression gives whatever the current environment's `setInterval` returns.

Common mistakes

- `useRef<HTMLInputElement>()` with no argument — an error with React 19 types (see §11.10).
- Declaring the wrong element type: `useRef<HTMLDivElement>(null)` on an `<input>` is a type error at `ref={...}`.
- Reading or writing `ref.current` **during render**. Refs are for event handlers and Effects (code inside `useEffect` that runs after React updates the screen, to sync with things outside React); React Compiler (a build tool that adds memoization — caching results so they are reused between renders — for you) and its lint rules (automatic checks that flag risky code patterns) flag render-time ref access (Chapter 10).

TL;DR

`useRef<HTMLXxxElement>(null)` for DOM, `useRef<T>(initial)` for values. In React 19 `ref` is a normal prop typed `Ref<T>`; use `useImperativeHandle` to expose a custom handle.

11.7 Typing custom Hooks and Context

Returning a tuple: `as const`

A Hook that returns `[on, toggle]` would be inferred as `(boolean | (() => void))[]` — an array where each element could be either, so `on` isn't known to be a boolean. `as const` makes it a **readonly tuple** `readonly [boolean, () => void]` (a tuple is a fixed-length array where each position has its own type; readonly means it can't be changed):

```
function useToggle(initial = false) {
  const [on, setOn] = useState(initial);
  const toggle = useCallback(() => setOn(v => !v), []);
  return [on, toggle] as const;
}
```

(Returning an **object** — `{ on, toggle }` — avoids the problem entirely and is easier to extend; prefer objects when a Hook returns more than two things.)

A generic Hook with a discriminated-union result

`code/05_hooks_and_context.tsx`:

```
type FetchState<T> =
  | { status: 'loading' }
  | { status: 'error'; error: string }
  | { status: 'success'; data: T };

function useJson<T>(url: string): FetchState<T> {
  const [state, setState] = useState<FetchState<T>>({ status: 'loading' });
  useEffect(() => {
    let ignore = false;
    setState({ status: 'loading' });
    fetch(url)
      .then(r => (r.ok ? (r.json() as Promise<T>) : Promise.reject(new Error(`HTTP ${r.status}`))))
      .then(data => { if (!ignore) setState({ status: 'success', data }); })
      .catch((e: unknown) => { if (!ignore) setState({ status: 'error', error: e instanceof Error ? e.message :
String(e) }); });
    return () => { ignore = true; };
  }, [url]);
  return state;
}
```

The caller picks `T` (`useJson<Todo>('/api/todos/1')`). Because the result is a union, the component *must* check `status` before touching `data`:

```
function TodoTitle() {
  const result = useJson<Todo>('/api/todos/1');
  const [details, toggle] = useToggle();
  const { user, logout } = useAuth();
  if (result.status === 'loading') return <p>loading...</p>;
  if (result.status === 'error') return <p>error: {result.error}</p>;
  // Here TypeScript knows result.data is a Todo.
  return (/* ... {user}: {result.data.title}{details && ` (#${result.data.id})`} ... */);
}
```

This is better than three separate fields (`data?`, `error?`, `loading`), which allow nonsense states such as "loading **and** has an error". Note `catch (e: unknown)`: in strict mode caught values are `unknown`, and you narrow with `instanceof Error` (a runtime check that the value was created from the `Error` class).

`as Promise<T>` is not validation. It tells the compiler "trust me" — if the API returns a different shape, TypeScript cannot know. At trust boundaries (the places where data from outside your code enters: network, `localStorage` — the browser's small key-value store that survives page reloads — URL params), validate with a schema library such as Zod, which both checks the data at runtime (while the program is actually running) and produces the TypeScript type. Libraries like TanStack Query (a library

that fetches, caches and refreshes server data) are generic in the same way (`useQuery<Todo>`), see Chapter 9.

Context that is never `null` for consumers

`createContext` needs a default value. When there's no sensible default, use `null` and a custom Hook that throws if the provider (the component that supplies the context value to everything inside it) is missing — callers then get the non-nullable type (a type that excludes `null`):

```
type Auth = { user: string; logout: () => void };
const AuthContext = createContext<Auth | null>(null);

function useAuth(): Auth {                        // callers get Auth, never null
  const ctx = useContext(AuthContext);
  if (!ctx) throw new Error('useAuth must be used inside <AuthProvider>');
  return ctx;
}
function AuthProvider({ children }: { children: ReactNode }) {
  const [user, setUser] = useState('ada');
  // (in a real app, memoize this object - see Chapter 10, section 10.10)
  return <AuthContext value={{ user, logout: () => setUser('guest') }}>{children}</AuthContext>;
}
```

Output (fetch is replaced by a local fake so the example runs offline; the second part renders `TodoTitle` without a provider):

```
loading...
guest: loaded from /api/todos/1 (#1)
without provider -> useAuth must be used inside <AuthProvider>
```

The alternative, `createContext<Auth>({} as Auth)`, compiles but lies: a component outside the provider would crash later with a confusing `undefined is not a function`.

TL;DR

Tuples need `as const` (or return an object). Generic Hooks take `<T>`; return discriminated unions for async state. Context: `createContext<T | null>(null)` + a `useX()` that throws.

11.8 Generic components

What it is

A **generic component** has a type parameter, like a generic function: `function List<T>(props: ListProps<T>)`. TypeScript **infers** `T` from the props you pass, then checks the other props against it. One `List` works for books, users or strings, and `renderItem` always receives the right item type.

`code/06_generic_components.tsx`:

```tsx
type ListProps<T> = {
  items: T[];
  getKey: (item: T) => Key;
  renderItem: (item: T) => ReactNode;
  emptyText?: string;
};
function List<T>({ items, getKey, renderItem, emptyText = 'Nothing here' }: ListProps<T>) {
  if (items.length === 0) return <p>{emptyText}</p>;
  return <ul>{items.map(item => <li key={getKey(item)}>{renderItem(item)}</li>)}</ul>;
}
// In an arrow function inside a .tsx file write <T,> so the parser doesn't read <T> as JSX:
const Count = <T,>({ items }: { items: T[] }) => <small>{items.length} items</small>;

// A select whose value type is a union of string literals, inferred from `options`
type SelectProps<V extends string> = {
  label: string;
  value: V;
  options: readonly { value: V; label: string }[];
  onChange: (value: V) => void;
};
function Select<V extends string>({ label, value, options, onChange }: SelectProps<V>) {
  return (
    <label>{label}{' '}
      <select value={value} onChange={e => onChange(e.target.value as V)}>
        {options.map(o => <option key={o.value} value={o.value}>{o.label}</option>)}
      </select>
    </label>
  );
}

// A table whose columns may only name real keys of the row type
type Column<Row> = { key: keyof Row & string; header: string };
function Table<Row extends { id: number }>({ rows, columns }: { rows: Row[]; columns: Column<Row>[] }) {
  // ... <th>{c.header}</th> ... <td>{String(r[c.key])}</td> ...
}

type Book = { id: number; title: string; year: number };
const SORTS = [{ value: 'title', label: 'Title' }, { value: 'year', label: 'Year' }] as const;

function App() {
  const [sort, setSort] = useState<'title' | 'year'>('title');
  const sorted = [...books].sort((a, b) => (sort === 'year' ? a.year - b.year : a.title.localeCompare(b.title)));
  return (<>
      <Select label="Sort by" value={sort} options={SORTS} onChange={setSort} />
      <List items={sorted} getKey={b => b.id} renderItem={b => `${b.title} (${b.year})`} />
      <List items={[] as string[]} getKey={s => s} renderItem={s => s} emptyText="No tags" />
      <Count items={books} />
      <Table rows={books} columns={[{ key: 'title', header: 'Title' }, { key: 'year', header: 'Year' }]} />
    </>);
}
```

Output (then changing the select to "year"):

```
list : Beloved (1987) | Dune (1965) | Neuromancer (1984)
year : Dune (1965) | Neuromancer (1984) | Beloved (1987)
other: No tags / 3 items
table: TitleYear | Dune1965 | Neuromancer1984 | Beloved1987
```

How it works

1. In `<List items={sorted} ...>`, TypeScript sees `items: Book[]`, so `T = Book`.
2. `renderItem={b => ...}` is then checked as `(item: Book) => ReactNode` — `b.title` autocompletes, `b.author` would be an error.
3. **Constraints** (`V extends string`, `Row extends { id: number }`) restrict what `T` may be, and let the component use those guarantees (`r.id` as a key).
4. `keyof Row & string` is the union of `Row`'s property names (`'id' | 'title' | 'year'`), so a column for a missing field is a compile error (§11.10, error 8).

5. `as const` on `SORTS` keeps the literal values `'title'` and `'year'`, so `Select` infers `V = 'title' | 'year'` and `onChange={setSort}` type-checks.

When not to use generics

If a component only ever handles one type, a plain props type is clearer. Generics pay off in reusable building blocks: lists, tables, selects, comboboxes (a text box with a dropdown of suggestions), form fields.

TL;DR

`function List<T>(props: ListProps<T>)` — T is inferred from props and flows into callbacks. Constrain with `extends`, use `keyof` for field names, `<T,>` for arrow functions in `.tsx`.

11.9 Discriminated unions for props

The problem it solves

Props like `{ status: string; error?: string; count?: number; onRetry?: () => void }` allow nonsense: a "success" message with an error, an "error" without a retry handler. Every consumer must guess which combinations are valid, and the component needs defensive checks. A **union of props types**, one per case, makes the invalid combinations **impossible to write**.

`code/07_union_props.tsx`:

```tsx
// 1. Each `status` value comes with exactly the props it needs
type StatusProps =
  | { status: 'loading' }
  | { status: 'error'; error: string; onRetry: () => void }
  | { status: 'success'; count: number };

function Status(props: StatusProps) {
  switch (props.status) {                    // narrow on the discriminant `status`
    case 'loading': return <p>Loading…</p>;
    case 'error':   return <p role="alert">{props.error} <button onClick={props.onRetry}>Retry</button></p>;
    case 'success': return <p>{props.count} results</p>;
  }
}

// 2. Either a link or a button
type LinkLike = { as: 'a'; href: string } & Omit<ComponentProps<'a'>, 'href'>;
type ButtonLike = { as?: 'button' } & ComponentProps<'button'>;
type ActionProps = (LinkLike | ButtonLike) & { children: ReactNode };

function Action(props: ActionProps) {
  if (props.as === 'a') {
    const { as: _as, ...rest } = props;       // here props is LinkLike
    return <a className="action" {...rest} />;
  }
  const { as: _as, ...rest } = props;         // here props is ButtonLike
  return <button className="action" type="button" {...rest} />;
}

// 3. "One or the other, not both": the `never` trick forbids the other prop
type ImageSource =
  | { src: string; icon?: never }
  | { icon: 'user' | 'star'; src?: never };
function Thumb(props: ImageSource & { alt: string }) {
  return props.src ? <img src={props.src} alt={props.alt} /> : <span aria-label={props.alt}>[{props.icon}]</span>;
}
```

Usage and result:

```
<Status status="loading" />
<Status status="error" error="Network down" onRetry={() => {}} />
<Status status="success" count={42} />
<Action as="a" href="/docs">Docs</Action>
<Action onClick={() => {}}>Save</Action>
<Thumb src="/a.png" alt="avatar" />
<Thumb icon="star" alt="favourite" />
```

Output:

```
<p>Loading…</p>
<p role="alert">Network down <button>Retry</button>
</p>
<p>42 results</p>
<a class="action" href="/docs">Docs</a>
<button class="action" type="button">Save</button>
<img alt="avatar" src="/a.png">
<span aria-label="favourite">[star]</span>
```

How it works

1. The shared literal field (`status`, `as`) is the **discriminant**.
2. At the call site, TypeScript picks the union member whose discriminant matches and checks the remaining props against it: `<Status status="success" />` without `count` fails (§11.10, error 6).
3. Inside the component, checking the discriminant (`switch`, `if`) narrows `props`, so `props.error` is only accessible in the `'error'` branch.
4. Don't destructure the props in the parameter list (`function Status({ status, error })`) — destructuring before narrowing loses the link between the fields. Narrow `props` first, then read fields. (Recent TypeScript versions can narrow destructured `const` variables in simple cases, but keeping `props` whole is the reliable pattern.)
5. `prop?: never` means "this prop must not be passed" — the standard way to express "exactly one of these".

Real-world examples

- Component libraries use `as`/`asChild` props or link-vs-button unions so a clickable thing renders correct, accessible HTML (markup that keyboard and screen-reader users can operate).
- Alerts/toasts (small pop-up messages that disappear on their own) with `variant` unions, form fields where `type: 'select'` requires `options`, modals (dialog boxes that sit on top of the page until closed) where `dismissible: true` requires `onClose`.

In the interview

30-second answer: "I type props with a type alias on the destructured parameter, `children` as `ReactNode`, and extend native elements with `ComponentProps<'button'>`. Handlers use React's event types such as `ChangeEvent<HTMLInputElement>`, though inline handlers are inferred. For state I let inference work and add generics for `null` or empty arrays. For variant props I use discriminated unions so invalid combinations don't compile, and reducers get action unions with a `never` exhaustiveness check. Reusable lists and tables are generic components."

- *`ReactNode` vs `ReactElement` vs `JSX.Element`?* — `ReactNode` is anything renderable (use for children); `ReactElement`/`JSX.Element` is only a JSX element.
- *How do you type `ref` in React 19?* — As a normal prop: `ref?: Ref<HTMLInputElement>`, or get it for free from `ComponentProps<'input'>`; `forwardRef` isn't needed.
- *Why not `any`?* — It disables checking and spreads; use `unknown` and narrow, or a proper type.

TL;DR

One union member per valid prop combination, sharing a literal discriminant; narrow `props` before reading fields; `?: never` forbids a prop.

11.10 Reading type errors, and common mistakes

TypeScript error messages look intimidating, but they follow one pattern: *"Type **A** (what you gave) is not assignable to type **B** (what was expected)"* ("not assignable" means a value of type A can't be used where B is required), often followed by indented lines that zoom in on the exact property that differs. Read the **last** indented line first — it's usually the real reason.

`code/bad_types.tsx` collects twelve common mistakes (it is excluded from the normal check and from the runner):

```
const e1 = <Badge />;                                  // 1. required prop missing
const e2 = <Badge label="x" tone="danger" />;           // 2. value outside the union

function Profile() {
  const [user] = useState<{ name: string } | null>(null);
  const [tags, setTags] = useState([]);
  setTags(['a']);                                      // 4. useState([]) without a type -> never[]
  return <p>{user.name} {tags.length}</p>;             // 3. user may be null
}

function Plan() {
  function handle(e: ChangeEvent<HTMLInputElement>) { console.log(e.target.value); }
  return <select onChange={handle} />;                 // 5. input handler on a <select>
}

const e6 = <Status status="success" />;                // 6. 'success' requires count
const e7 = <Thumb src="/a.png" icon="star" />;          // 7. both src and icon
const e8 = <Table rows={[{ id: 1, title: 'Dune' }]} columns={[{ key: 'author', header: 'Author' }]} />;  // 8. no such key

type Action = { type: 'add'; sku: string } | { type: 'clear' };
function reducer(state: string[], action: Action): string[] {
  switch (action.type) {
    case 'add': return [...state, action.sku];
    default: { const unreachable: never = action; return unreachable; }   // 9. 'clear' not handled
  }
}
function Cart() { const [items, dispatch] = useReducer(reducer, []); dispatch({ type: 'remove' }); return <p>{items}</p>; }  // 10. unknown action

function Focus() {
  const ref = useRef<HTMLInputElement>();              // 11. React 19 types: useRef needs an argument
  return <input ref={ref} />;
}

const e12 = <Box>{{ title: 'not renderable' }}</Box>;   // 12. a plain object is not a ReactNode
```

Output of `tsc --noEmit -p tsconfig.bad.json` (real, unedited):

```
bad_types.tsx(7,13): error TS2741: Property 'label' is missing in type '{}' but required in type 'BadgeProps'.
bad_types.tsx(8,29): error TS2322: Type '"danger"' is not assignable to type '"error" | "info" | "warning" |
undefined'.
bad_types.tsx(13,12): error TS2322: Type 'string' is not assignable to type 'never'.
bad_types.tsx(14,14): error TS18047: 'user' is possibly 'null'.
bad_types.tsx(19,18): error TS2322: Type '(e: ChangeEvent<HTMLInputElement, Element>) => void' is not assignable
to type 'ChangeEventHandler<HTMLSelectElement, HTMLSelectElement>'.
  Types of parameters 'e' and 'event' are incompatible.
    Type 'ChangeEvent<HTMLSelectElement, HTMLSelectElement>' is not assignable to type
'ChangeEvent<HTMLInputElement, Element>'.
      Type 'HTMLSelectElement' is missing the following properties from type 'HTMLInputElement': accept, align,
alt, capture, and 38 more.
bad_types.tsx(24,13): error TS2322: Type '{ status: "success"; }' is not assignable to type 'IntrinsicAttributes &
StatusProps'.
  Property 'count' is missing in type '{ status: "success"; }' but required in type '{ status: "success"; count:
number; }'.
bad_types.tsx(28,32): error TS2322: Type 'string' is not assignable to type '"star" | "user" | undefined'.
bad_types.tsx(32,65): error TS2322: Type '"author"' is not assignable to type '"id" | "title"'.
bad_types.tsx(38,22): error TS2322: Type '{ type: "clear"; }' is not assignable to type 'never'.
bad_types.tsx(41,81): error TS2322: Type '"remove"' is not assignable to type '"add" | "clear"'.
bad_types.tsx(44,15): error TS2554: Expected 1 arguments, but got 0.
bad_types.tsx(49,21): error TS2353: Object literal may only specify known properties, and 'title' does not exist
in type 'Iterable<ReactNode> | Promise<AwaitedReactNode> | ReactElement<unknown, string |
JSXElementConstructor<any>> | ReactPortal'.
```

Decoding each error

#	Error says	What it means	Fix
1	`Property 'label' is missing`	required prop not passed	pass it, or make it optional with a default
2	`"danger"` not assignable to the union	typo or unsupported variant	use an allowed value, or extend the union
3	`'user' is possibly 'null'`	state can still be `null`	narrow first: `user ? ... : ...` or `user?.name`
4	`string` not assignable to `never`	`useState([])` inferred `never[]`	`useState<string[]>([])`
5	handler for `HTMLInputElement` on a `<select>`	wrong element type in the event	`ChangeEvent<HTMLSelectElement>` (note: in `@types/react` 19 the event types have a second type parameter for `target`, which you normally leave out)
6	`Property 'count' is missing ... required in type '{ status: "success"; count: number; }'`	the chosen union member needs `count`	pass `count`
7	`string` not assignable to `"star" | "user" | undefined` on `icon`	with `src` given, TypeScript tried the members and none fits both props — the message points at one of them	pass only one of `src` / `icon`
8	`"author"` not assignable to `"id" | "title"`	column names a field the row type doesn't have	fix the key, or add the field to the data
9	`{ type: "clear" }` not assignable to `never`	exhaustiveness check caught an unhandled action	add `case 'clear'`
10	`"remove"` not assignable to `"add" | "clear"`	dispatching an action the reducer doesn't know	dispatch a valid action or add it to the union
11	`Expected 1 arguments, but got 0`	React 19 types require an initial value for `useRef`	`useRef<HTMLInputElement>(null)`
12	`'title' does not exist in type ... ReactNode ...`	rendering a plain object as a child (would also crash at runtime)	render a field: `{post.title}`

Other common mistakes

- **`as` everywhere.** `as Something` silences the checker without checking. Use it only when you truly know more than the compiler (as in `e.target.value as V` above), never to hide a real mismatch.
- **Non-null assertions** `user!.name` (the `!` tells TypeScript "trust me, this isn't null" without any check) — same risk as `as`; narrow instead.
- **Duplicating types that already exist.** Use `ComponentProps<typeof X>`, `ReturnType<typeof fn>`, `Parameters<...>`, or export the props type from the component file.
- **Typing API responses by hand and trusting them** — validate at the boundary (Zod or similar) and derive the type from the schema.
- **Over-engineering generics** — a component used for one type doesn't need them.

Self-check

1. Why does `useState([])` cause trouble? → The type is inferred as `never[]`, so nothing can be added.
2. Which type do you use for `children`? → `ReactNode`.
3. How do you get all the props of a native `<input>`? → `ComponentProps<'input'>`.

4. How do you make a reducer fail to compile when an action isn't handled? → Assign `action` to a `never` variable in `default`.
5. In React 19, how does a function component receive a ref? → As a regular `ref` prop typed `Ref<T>`.
6. Does `tsc` run when Vite builds your app? → No; the bundler only strips types. Run `tsc --noEmit` separately (CI or build script).

Chapter 11 TL;DR

Idea	Remember
Setup	`.tsx`, `@types/react(-dom)`, `strict: true`, `jsx: "react-jsx"`; bundler strips types, `tsc --noEmit` checks them
Props	`function C({ a, b = 1 }: Props)`; `?` optional; literal unions for variants; function types for callbacks
Children	`ReactNode`; `PropsWithChildren<P>`; native props via `ComponentProps<'button'>`
Events	Inline = inferred; otherwise `ChangeEvent<HTMLInputElement>`, `SubmitEvent<HTMLFormElement>` (older: `FormEvent`), `MouseEvent<...>`, `KeyboardEvent<...>` from `react`
State	Infer; `useState<T | null>(null)`, `useState<string[]>([])`, literal unions
Reducers	Action = discriminated union; `never` exhaustiveness check
Refs	`useRef<HTMLXxxElement>(null)`; argument required in React 19 types; `ref` is a prop typed `Ref<T>`; `useImperativeHandle` for custom handles
Hooks & Context	`as const` tuples or objects; generic Hooks with union results; `createContext<T | null>(null)` + throwing `useX()`
Generic components	`List<T>` infers `T` from props; constraints with `extends`; `keyof`; `<T,>` in arrow functions
Union props	One member per valid combination; narrow before destructuring; `?: never` to forbid
Errors	Read "A is not assignable to B" bottom-up; avoid `any`, `as`, `!` as escape hatches

Next: Chapter 12 — Testing React.